AI-BASED TWO-WHEELER DIAGNOSTIC AND WORKSHOP MANAGEMENT PORTAL

(BIKECARE AI)

A Project Report Submitted in partial fulfillment of the
requirements for the award of the Degree of

Bachelor of Science (Computer Science)

By

SAMIR UTTAM NAUKUDKAR

Roll no: - 55

Under the Guidance of

Prof. Vaishali Desai

DEPARTMENT OF COMPUTER SCIENCE

LORD UNIVERSAL COLLEGE

(Affiliated to University of Mumbai)

Topiwala Marg, Off Station Road, Goregaon (W), Mumbai – 400104

Academic Year

2026-27

LORD UNIVERSAL COLLEGE

DEPARTMENT OF COMPUTER SCIENCE

Topiwala Marg, Off Station Road, Goregaon (West), Mumbai – 400104
(Affiliated to the University of Mumbai)

CERTIFICATE

This is to certify that the project entitled "AI-BASED TWO-WHEELER DIAGNOSTIC AND WORKSHOP MANAGEMENT PORTAL (BIKECARE AI)" is a bona fide work of Mr. SAMIR UTTAM NAUKUDKAR (Roll No: 55, Seat No: CS-2026-055) submitted to the University of Mumbai in partial fulfillment of the requirements for the award of the Degree of Bachelor of Science in Computer Science (Semester VI) during the academic year 2026–2027.

Prof. Vaishali Desai
Project Guide
Department of Computer Science

Head of Department
Department of Computer Science
Lord Universal College

External Examiner
University of Mumbai

Dr. J. K. Sangle
Principal
Lord Universal College, Mumbai

DECLARATION

I, Mr. SAMIR UTTAM NAUKUDKAR, Student of LORD UNIVERSAL COLLEGE, studying in T.Y. B.Sc. COMPUTER SCIENCE, hereby declare that I have completed the Project entitled AI-BASED TWO-WHEELER DIAGNOSTIC AND WORKSHOP MANAGEMENT PORTAL (BIKECARE AI) during the academic year 2026-2027.

The report work is original, and the information and data included in the report are true and based on the system design, database structure, application development, implementation, and testing carried out as part of the BikeCare AI project.

Signature of the Student with Date: _____

Name of Student: Samir Uttam Naukudkar

Roll No: 55

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who has supported and guided me in the successful completion of my project entitled “AI-BASED TWO-WHEELER DIAGNOSTIC AND WORKSHOP MANAGEMENT PORTAL (BIKECARE AI).”

First and foremost, I would like to thank LORD UNIVERSAL COLLEGE for providing me with the necessary facilities and a supportive learning environment to complete this project successfully.

I would like to express my special and sincere thanks to my project guide, Prof. Vaishali Desai, for her valuable guidance, encouragement, suggestions, and continuous support throughout the development of this project. Her knowledge and constructive feedback helped me improve my understanding and successfully complete the project.

I am also thankful to the Principal and the Department of Computer Science for providing me with the opportunity and resources required to carry out this project.

Finally, I would like to express my heartfelt thanks to my parents, friends, and classmates for their constant encouragement, support, and motivation throughout the completion of this project.

ABSTRACT

The BikeCare AI – Smart Two-Wheeler Diagnostic & Workshop Management Portal is a web-based application designed to simplify and improve the process of diagnosing mechanical issues, booking service packages, and managing motorcycle repair operations. In urban areas, the increasing number of two-wheelers and limited transparency in traditional neighborhood garages often result in diagnostic guesswork, arbitrary billing, time wastage, and difficulty in tracking live repair progress. The proposed system provides a convenient digital platform through which users can run AI-powered sound and vibration diagnostics, select transparent service packages, book doorstep pickup, and track their digital job cards in real time.

The system allows workshop administrators and mechanics to add and manage service packages, monitor active repair bays, assign incoming bookings to specific mechanics, and update job card milestone stages from inspection to delivery. Users can register and log in to the system, describe mechanical symptoms and acoustic noises, select their preferred service package and appointment slot, and confirm their booking. The system maintains real-time booking status progression to ensure transparency and streamline garage throughput.

The main objective of the BikeCare AI portal is to provide a user-friendly, efficient, and organized solution for two-wheeler servicing. By eliminating guesswork through intelligent AI symptom cross-referencing and providing centralized booking and workshop management facilities, the system helps save time, prevents unexpected breakdowns, and improves the overall customer care experience. The project demonstrates how modern information technology and artificial intelligence can be combined to build a scalable automotive maintenance ecosystem.

TABLE OF CONTENTS

SR NO / Section	Contents/Chapters	Page no
	Chapter 1- Problem Identification	11
1.1	Problem Statement	11
1.2	Problem Justification and Scope Definition	11
1.3	Stakeholder Identification	13
1.4	Feasibility Analysis	14
1.5	Use-Case Analysis	17
1.6	Requirement Prioritization	18
1.7	Constraints and Assumptions	19
	Chapter 2 - Software Development Life Cycle (SDLC) Planning:	21
2.1	Selection of SDLC Model	21
2.2	Work Breakdown Structure (WBS)	21
2.3	Project Timeline (Gantt Chart)	22
2.4	Resource Requirement	22
	Chapter 3 - System Modeling using UML	24
3.1	ER Diagram	24
3.2	Use Case Diagram	25
3.3	Class Diagram	26
3.4	Sequence Diagram	27
3.5	Activity Diagram	28
3.6	Deployment Diagram	30
	Chapter 4 – System Architecture Design	31

SR NO	Contents/Chapters	Page no
4.1	Frontend Architecture	31
4.2	Backend Architecture	31
4.3	Data Schema Design	32
	Chapter 5 - Implementation	33
5.1	Frontend Implementation	33
5.2	Backend Implementation	38
5.3	Database Implementation	41
	Chapter 6 – Integration and System Testing	45
6.1	Unit Testing	45
6.2	Black-Box Testing	46
6.3	Integration Testing	49
6.4	Test Case Preparation	51
6.5	Bug Testing	53
	Conclusion	55
	Future Scope	56
	References	57

LIST OF FIGURES

Sr No	Figure Title	Page No
2.1	Work Breakdown Structure (WBS)	22
2.2	Project Timeline (Gantt Chart)	22
3.1	ER Diagram	24
3.2	Use Case Diagram	25
3.3	Class Diagram	26
3.4	Sequence Diagram	27
3.5	Activity Diagram	28
3.6	Deployment Diagram	30
4.1	Frontend architecture	31
4.2	Backend architecture	31
4.3	Data Schema Design	32
5.1	Home Page	33
5.2	Login Page	34
5.3	AI Diagnostics Page	35
5.4	Service Booking Page	37
5.5	Payment Signature Verification	39
5.6	Automated Email Receipt	40
5.7	Database Output	44
6.1.1	User Registration Test	46
6.1.2	User Login Test	46
6.2.1	Invalid Login Test	48

Sr No	Figure Title	Page No
6.2.2	Empty slot Search test	48
6.2.3	Invalid Location Search Test	49
6.3.1	Register Login	50
6.3.2	Dashboard	51
6.5.1	Input Command Error During Test Execution	54

LIST OF TABLES

Sr No	Table Title	Page no
1.1	Stakeholder Identification	13
1.2	Cost analysis	15
1.3	Requirement Prioritization	18
1.4	Types of Constraints	19
2.1	Resource Requirement	22
6.1	Unit Testing	45
6.2	Black-Box Testing	47
6.3	Integration Testing	49
6.4	Test Case Preparation	51
6.5	Bug Tracking	53

Chapter 1:

1.Problem Identification & Feasibility Study

1.1 Problem Statement:

Two-wheeler owners often face difficulties in diagnosing and repairing mechanical issues efficiently due to manual guesswork, unstandardized pricing, and paper-based workshop records. A Smart Two-Wheeler Diagnostic & Workshop Portal provides a centralized platform where riders can analyze symptoms, book verified service packages, make payments, and track live repair progress digitally. This improves accuracy, reduces waiting time, eliminates cost disputes, and ensures transparency across garage operations.

Need for the Project

1) To simplify the diagnosis and booking of two-wheeler repair packages.
2) To reduce the time required for identifying mechanical faults and reserving service bays.
3) To improve communication between riders, certified mechanics, and workshop managers.
4) To maintain accurate, organized, and secure digital job card records.
5) To minimize manual errors and enhance the overall two-wheeler maintenance experience.

1.2 Problem Justification and Scope Definition Justification

Many motorcycle riders still search for trusted garages manually by visiting local workshops or relying on unverified word-of-mouth recommendations, which often leads to delays, overcharging, repeated engine issues, and difficulty in scheduling repairs. Manual workshop processes also make it challenging to monitor mechanic bay availability, maintain accurate parts replacement records, and ensure transparent billing.

The BikeCare AI system addresses these issues by providing a centralized web platform where users can diagnose vehicle faults, book standardized service packages, manage appointments, and track digital job cards efficiently. The system improves workshop operations by reducing administrative effort, increasing diagnostic precision, enhancing transparency, and maintaining organized service histories.

Scope of the Project

The scope of the BikeCare AI – Two-Wheeler Diagnostic and Workshop Management Portal is to develop a centralized web-based platform that enables motorcycle riders and workshop operators to efficiently diagnose, book, manage, and track vehicle repairs. The system will streamline the service reservation process, maintain organized digital job cards, and improve communication between commuters, bay technicians, and garage managers. It will help eliminate acoustic guesswork, minimize billing errors, and monitor service bay workload in real time.

1) User Registration and Login: Users can create accounts and log in securely.

2) Profile Management: Users can manage personal contact and vehicle profile information.

3) AI Diagnostic Scanner: Users can input symptoms and audio noise descriptions for instant fault classification.

4) Service Package Booking: Users can select tiered packages (Express, Master, EV) with upfront pricing.

5) Booking Management: Users can view, edit, update, and manage service bookings prior to confirmation.

6) Live Job Card Tracking: Users can monitor active repair status across 6 progressive milestone phases.

7) Workshop Admin & Mechanic Portal: Mechanics and managers securely log in to /admin to dispatch jobs and log notes.

8) Admin Operations Panel: Admin can manage users, assign bay mechanics, review revenue, and audit system records.

9) Responsive Design: The website will be fully accessible on desktops, tablets, and mobile devices.

10) Online Payment & Verification: Integration of digital payment tracking to process service fees securely.

Out of Scope

1) Construction or physical metallurgical fabrication of motorcycle spare parts.

2) Integration with banking credit card merchant clearing houses.

3) Physical fabrication of hardware wireless OBD-II Bluetooth scanning dongles.

4) Direct real-time chat or instant messaging between drivers and mechanics.

5) Multi-city nationwide franchise garage chain network management.

6) Integration with external legacy ERP or corporate accounting mainframes.

7) Real-time GPS van navigation or satellite vehicle tracking.

1.3 Stakeholder Identification

Stakeholder	Role
Rider / Vehicle Owner	Search and diagnose bike issues, select service packages, book doorstep pickup, track repair milestones, and make payments.
Workshop Bay Mechanic	Receive assigned job cards, inspect physical vehicle parts, log diagnostic findings, append extra parts costs, and complete repairs.
Workshop Administrator	Manage users, assign incoming bookings to available mechanics, oversee bay capacity, and audit customer reviews.
Management	Monitor workshop activities, review monthly revenue and booking reports, analyze bay throughput, and make business decisions.
System Administrator	Maintain the website, database persistence, security credentials, system performance, backups, and technical support.

Table 1.1

1.4 Feasibility Analysis

Technical Feasibility

The BikeCare AI system is technically feasible because the required technologies, tools, and resources are easily available. The project can be developed using standard web development technologies such as HTML5, CSS3, JavaScript/TypeScript, React, Node.js/Express, Python Flask, and JSON/MySQL database persistence. These technologies are reliable, cost-effective, and suitable for developing a secure and user-friendly system that allows users to diagnose vehicle faults, book packages, and track repair operations efficiently.

Considerations

1. The system should be developed using suitable web technologies that provide high performance and security.
2. A well-structured database should be used to store user, vehicle, package, and booking information efficiently.
3. The website should have a simple, responsive, and user-friendly interface that works across different devices.
4. Proper user authentication and role-based access control should be implemented to ensure data security.
5. The system should allow users to search, book, update, and track service cards efficiently.
6. The application should be scalable so that additional features and users can be supported in the future.
7. Regular maintenance, security updates, and database backups should be performed to ensure reliable system performance.

Economic Feasibility

Determines whether the project is financially viable.

Cost Analysis

Item	Estimated Cost
Software	¹0 (Open Source Tools)
Hardware	¹0 (Existing Device)
Cloud Hosting	¹0 (Developer Free Tier)
Development Cost	¹0 (Student Project)
Total Cost	¹0

Table 1.2

Operational Feasibility

Operational feasibility determines whether the BikeCare AI portal can be successfully implemented and used by motorcycle riders and workshop operators.

Considerations

- &Uuser Acceptance: Riders and workshop supervisors should be willing to adopt and use the system effectively.
- &Training Requirements: Basic intuitive interface eliminates complex training requirements for workshop staff.
- &Maintenance Requirements: Self-contained data persistence and automated heuristic fallbacks ensure smooth operation.
- &Organizational Support: Management should provide necessary shop-floor support and bay allocation.

Requirement Engineering

Functional Requirements

These define what the system should do.

1) User Registration and Login

2) User Profile Management

3) Bike Symptom & Noise Description Input

4) AI Diagnostic Scanner Execution

5) Service Package Selection & Price Configurator

6) Doorstep Booking Registration with Tracking Code

7) Update and Cancel Service Bookings

8) Workshop Admin & Mechanic Passcode Login (/admin)

9) Job Card Milestone State Progression

10) Mechanic Bay Workload Dispatching

11) Technician Diagnostic Notes & Extra Parts Billing

12) Online Payment Verification & Receipt Dispatch

Non-Functional Requirements

These define the quality attributes of the system.

- &#Performance: The system should process diagnostic requests quickly and provide sub-2.5 second response times.
- &#Security: Passcode authorization, route isolation, and data protection should be strictly implemented.
- &#Reliability: The system should operate consistently with minimal downtime and automated heuristic fallbacks.
- &#Usability: The interface should be simple, high-contrast, user-friendly, and easy to navigate.
- &#Scalability: The architecture should support an increasing number of riders and service bay bookings.
- &#Availability: The system should be accessible whenever authorized riders and mechanics need it.
- &#Maintainability: The application should be modular and easy to maintain, update, and enhance.

- Compatibility: The website should work properly on different modern browsers and mobile devices.

1.5 Use-Case Analysis

Use-case analysis identifies the interactions between users (actors) and the BikeCare AI System.

Components of a Use Case Actor

A person or external entity interacting with the application.

Use Case

A specific functionality provided by the system.

Example: Actors

- Motorcycle Rider / Vehicle Owner
- Workshop Mechanic / Bay Technician
- Workshop Service Administrator
- AI Diagnostic Engine (Gemini)

5.1 Use Cases

1) User Login & Authentication

2) Register Rider Account

3) Manage Motorcycle Profile

4) Run AI Acoustic Diagnostic Scan

5) View Standardized Packages & Pricing

6) Book Doorstep Service Pickup

7) Confirm and Track Booking Milestones

8) Manage Workshop Service Bays

9) Workshop Admin / Mechanic Passcode Login

10) Track Digital Job Card Status (BK-XXXX)

11) Generate Booking & Revenue Reports (Admin)

12) Online Payment Verification & Invoicing

1.6 Requirement Prioritization

Requirement prioritization helps determine which requirements are most important and should be implemented first.

Priority	Description
Must Have	Essential requirements that are necessary for the basic functioning of the system
Should Have	Important requirements that improve the system but are not mandatory.
Could Have	Desirable features that can be added to enhance user experience.
Won’t Have	Features that are deferred and planned for future versions.

Table 1.3

Example:-

Requirement	Priority
User Registration and Login	Must Have
AI Diagnostic Scanner	Must Have
Book Service Package	Must Have
View Job Card Details	Must Have
Booking Confirmation	Must Have

Requirement	Priority
Workshop Admin / Mechanic Login	Should Have
Vehicle Profile Management	Should Have
Service Bay Workload Management	Should Have
Live Booking Status Tracking	Should Have
Notification System	Could Have
Workshop Revenue Reports	Could Have
AI Automated Demand Forecasting	Won’t Have
Online Payment Integration	Should Have

1.7 Constraints and Assumptions Constraints

Constraints are limitations under which the system must operate.

Types of Constraints

Type	Example
Budget Constraint	The project is developed using free and open-source tools with minimal cost.
Time Constraint	The project must be completed within the academic college timeline.
Technology Constraint	The system will be developed using modern web technologies such as HTML, CSS, JavaScript, React, Node.js, and JSON/MySQL.
Hardware Constraint	Requires a computer or smartphone with an active internet connection for development and usage.
Legal Constraint	User contact and booking records must be protected according to privacy guidelines.

Type	Example
Security Constraint	The system must prevent unauthorized access and ensure secure admin passcode authentication.
Performance Constraint	The website should provide fast response times and support concurrent user interactions.

Table 1.4

Assumptions

Assumptions are conditions believed to be true during project planning.

Example Statement:

It is assumed that all motorcycle owners and workshop staff have basic knowledge of using computers or smartphones and have access to an active internet connection. Users are expected to enter accurate vehicle specifications, odometer mileage, and observable acoustic symptoms during diagnostic booking.

Chapter 2:

Software Development Life Cycle (SDLC) Planning:

2.1 Selection of SDLC model: - Waterfall Model

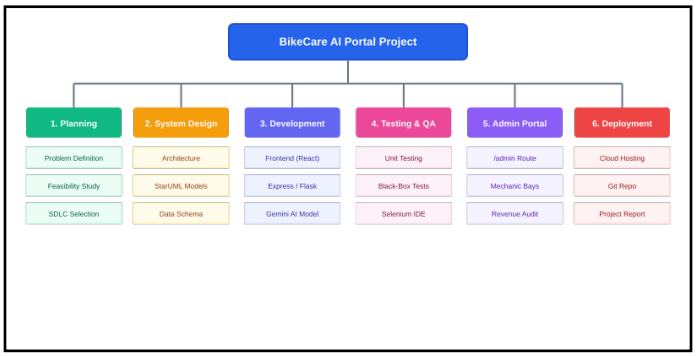
Selected Model: Waterfall SDLC Model

Reason for Selection: The Waterfall Model is selected for the BikeCare AI portal because the project requirements are clear, structured, and fixed from the beginning. It follows a simple and sequential development process, making it easy to plan, develop, and test each phase. The model provides proper documentation, which is essential for college project submissions.

Why Waterfall Model is best?

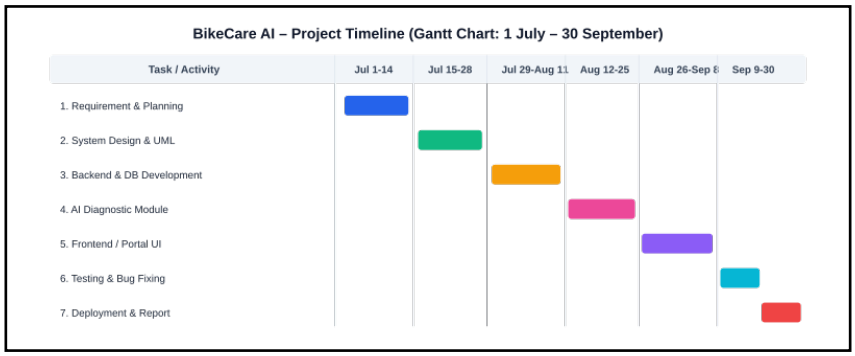
- Clear and fixed project requirements.
- Simple and easy to understand.
- Sequential development process.
- Easy to implement for students.
- Suitable for small and medium-sized projects.
- Provides proper documentation at every phase.
- Makes testing and debugging easier.
- Helps with better project planning and time management.

2.2 Work Breakdown Structure (WBS): - Figure 2.1



Work Breakdown Structure (WBS) – Smart Two-Wheeler Portal

2.3 Project Timeline (Gantt Chart): -Figure 2.2



Project Timeline (Gantt Chart) – Milestone Schedule

2.4 Resource Requirements: -Table 2.1

Resource Type	Required Resources	Purpose
Human Resources	Project Guide, Project Developer	To plan, design, develop, test, and manage the BikeCare AI System.
Hardware Resources	Laptop/Desktop, Internet Connection	Used for coding, database management, testing, and accessing online resources.

Resource Type	Required Resources	Purpose
Software Resources	Visual Studio Code, Node.js, Python, Git, Web Browser	Used for coding, server management, testing, debugging, and version control.
Technology	HTML, CSS, JavaScript, React, Tailwind, Express, Gemini AI	Used to develop the front-end, back-end, and AI engine of the BikeCare AI System.
Time Resources	12 Weeks	Estimated time required to complete the project, including planning, development, testing, and documentation.
Financial Resources	Domain Name, Web Hosting, Internet Charges (Optional for Deployment)	Required only if the project is deployed online for demonstration or public access.

Chapter 3 - System Modeling using UML

3.1 Entity-Relationship (ER) Diagram

The ER diagram models the persistent data structure of the system. It identifies entities, their attributes, primary/ foreign keys, and the cardinality of relationships between entities.

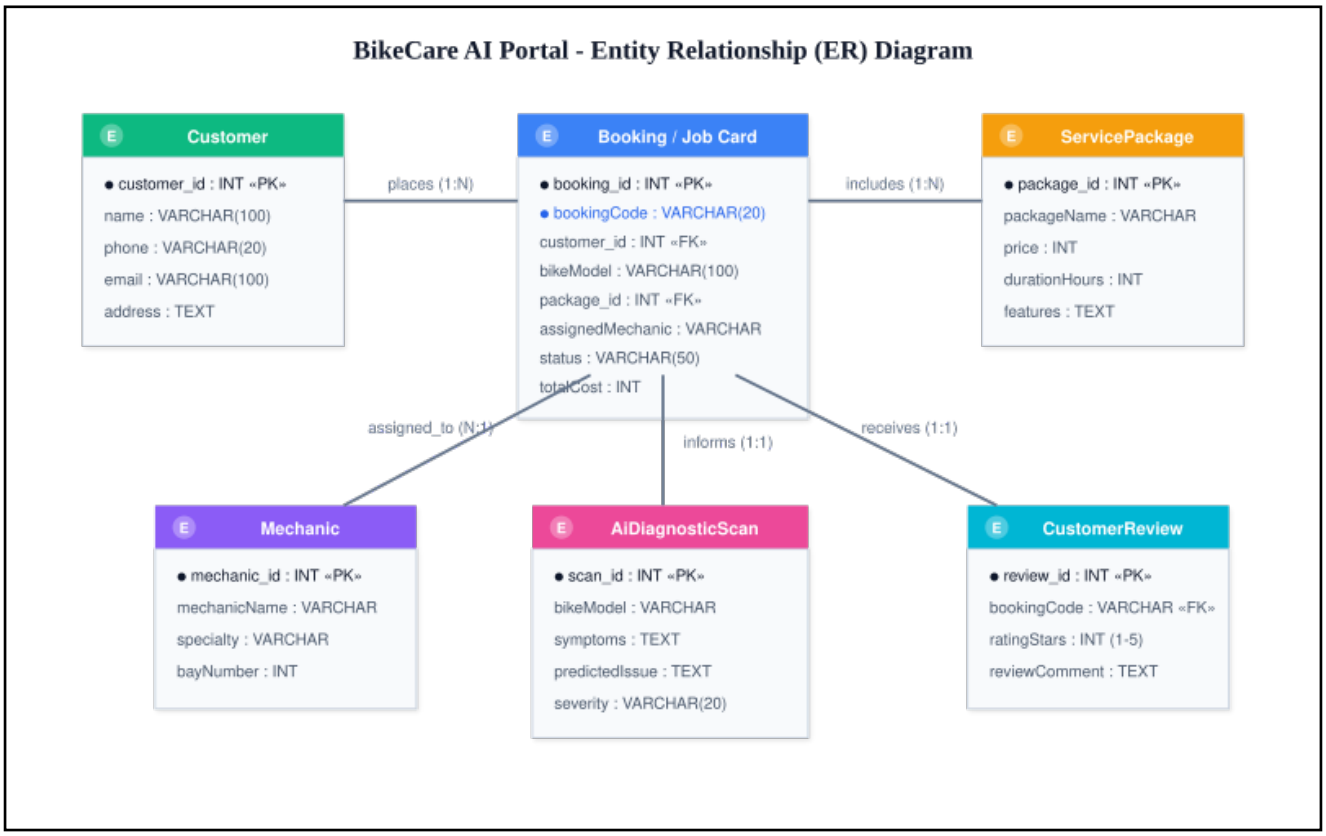


Figure 3.1 Entity-Relationship Diagram of the BikeCare AI System

3.2 Use Case Diagram

The use case diagram identifies the actors that interact with the BikeCare AI System and the functionalities (use cases) available to each actor.

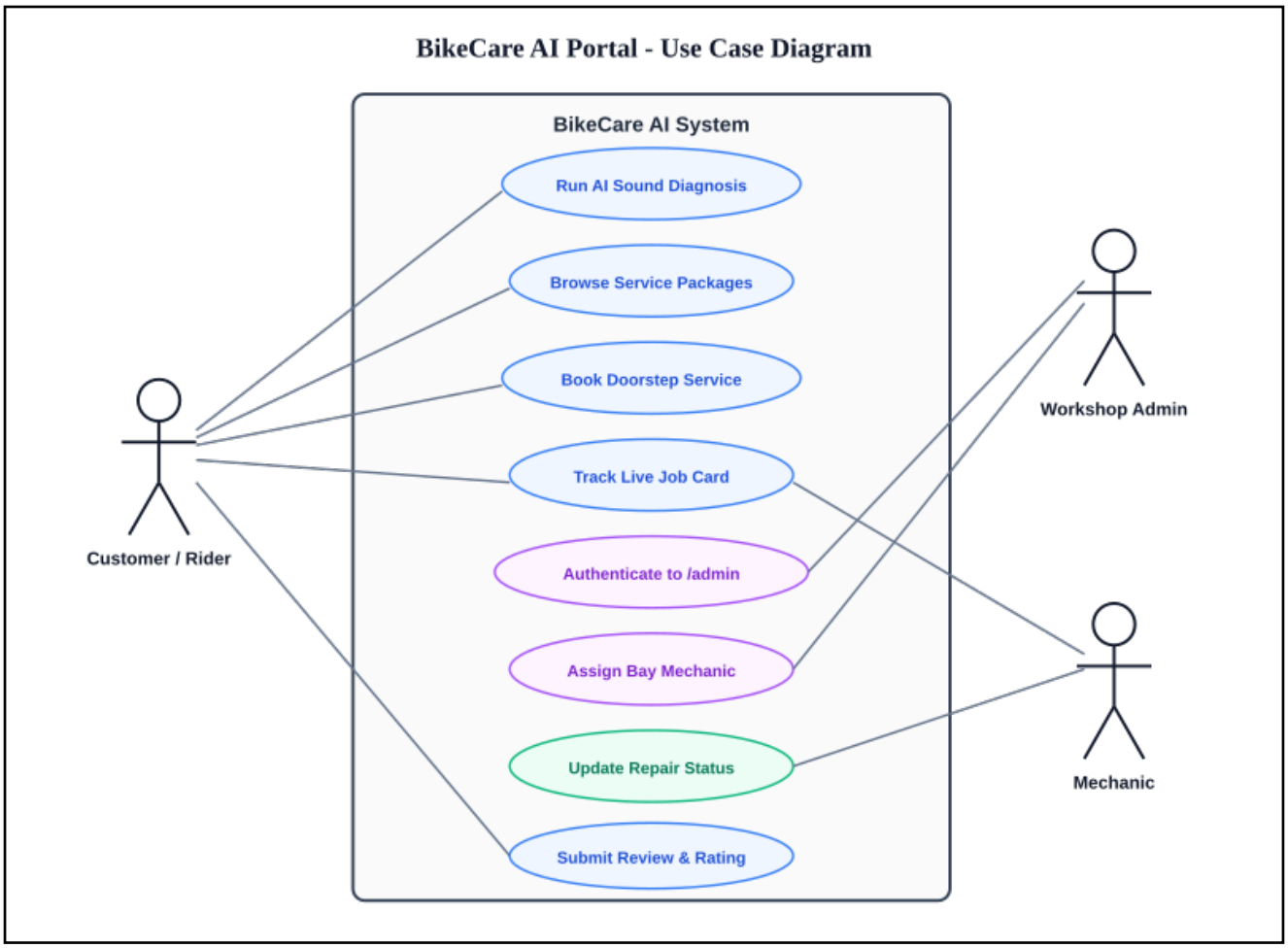


Figure 3.2 Use Case Diagram of the BikeCare AI System

3.3 Class Diagram

The class diagram represents the static structure of the system, showing the classes, their attributes, operations, and the relationships (association, aggregation) between them.

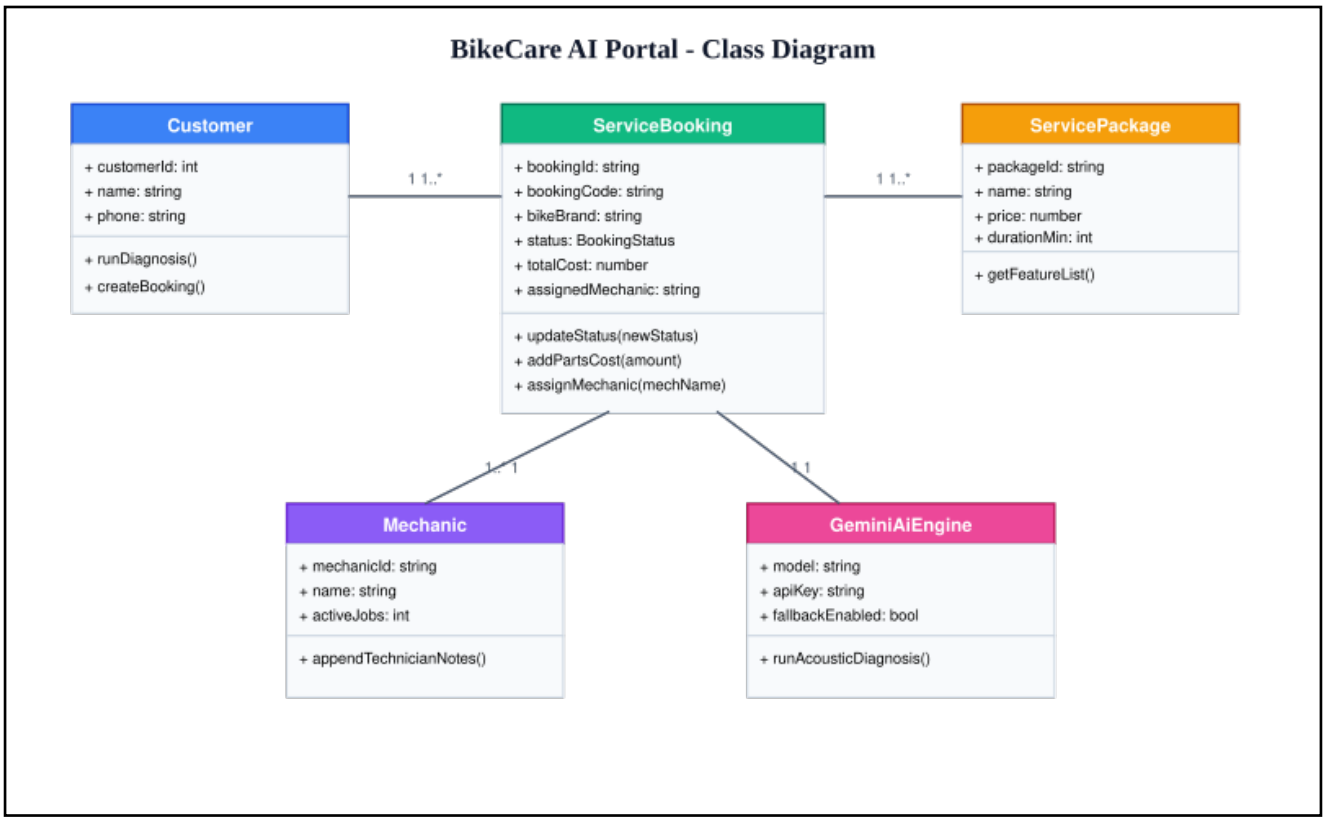


Figure 3.3 Class Diagram of the BikeCare AI System

3.4 Sequence Diagram

The sequence diagram illustrates the time-ordered interaction between objects for the primary scenario: a rider booking a service package. It shows message flow between the Rider, Mobile/Web App, Diagnostic Engine, Database, and Workshop Dispatcher, from symptom input through booking confirmation.

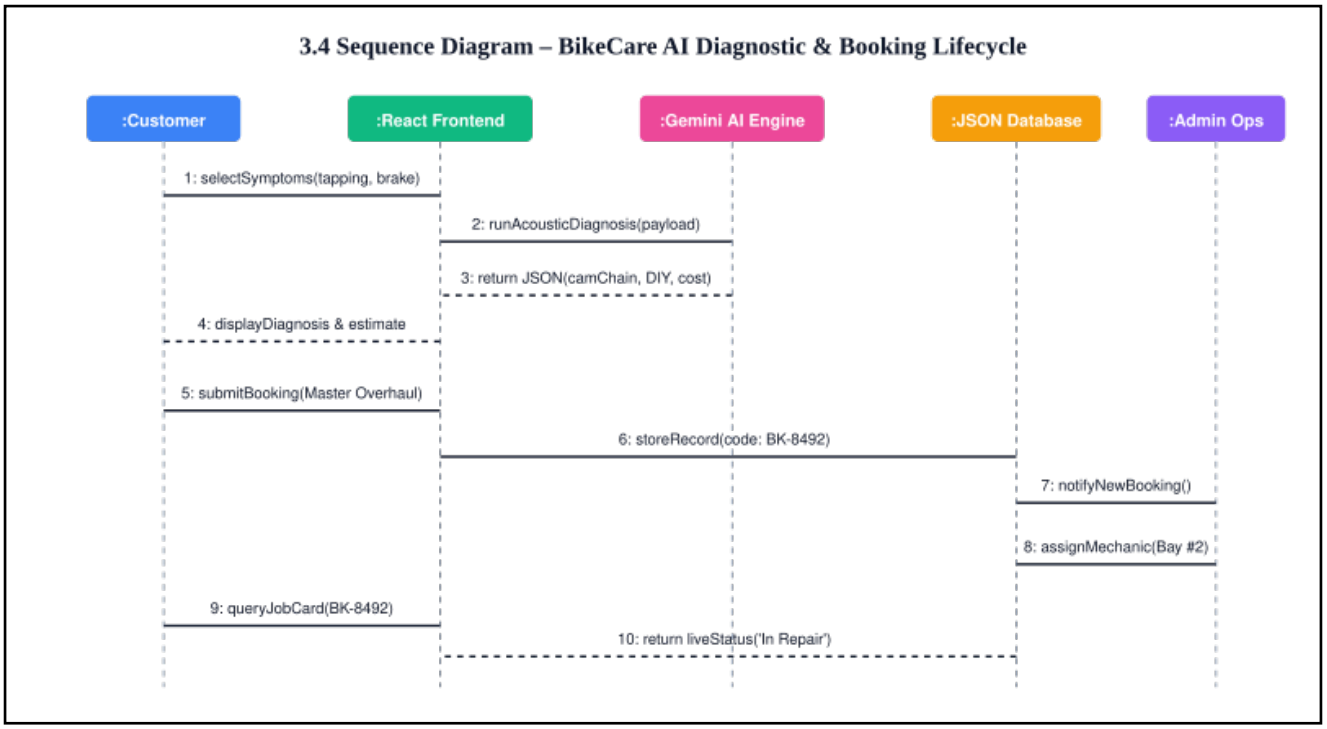
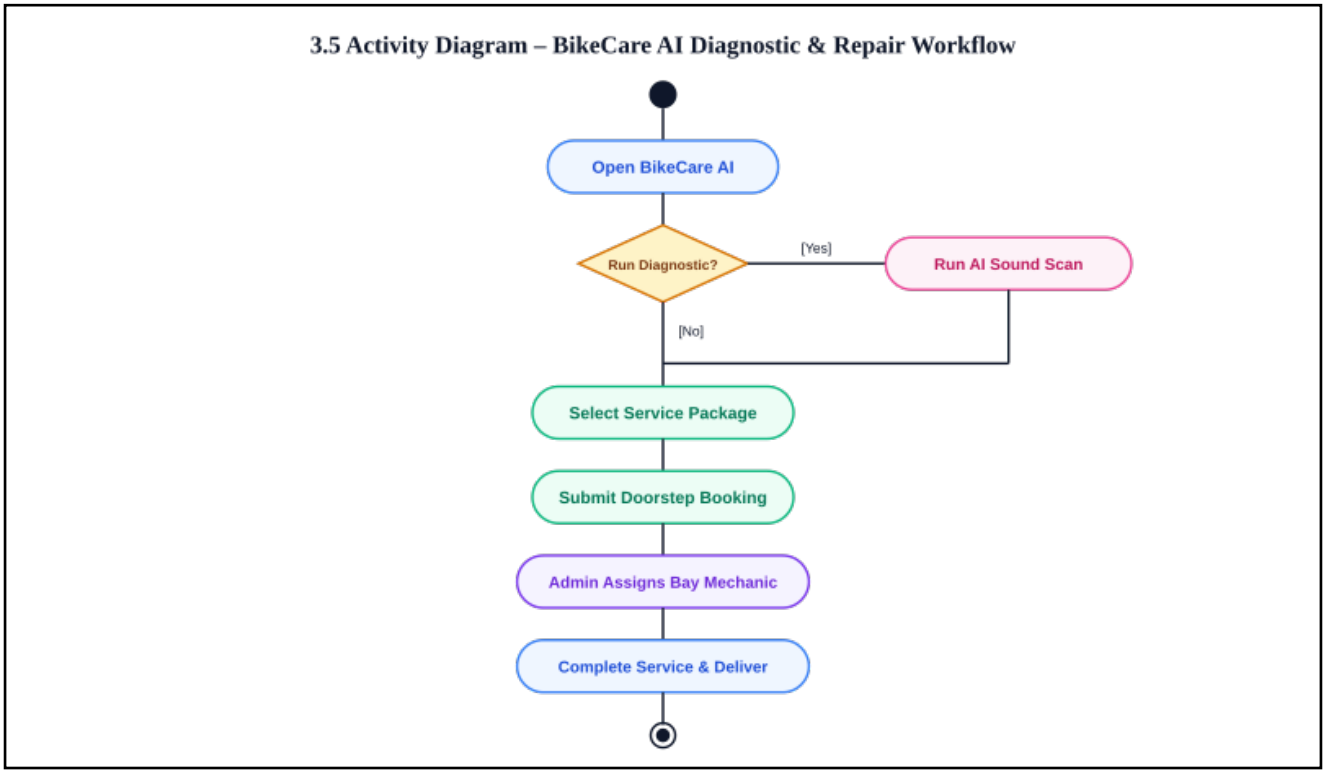


Figure 3.4 Sequence Diagram for the Diagnostic & Booking Process

3.5 Activity Diagram

The activity diagram models the complete workflow of the booking-to-delivery process, including decision points such as symptom match, package selection, payment success/failure, and mechanic assignment. Due to its length, the workflow is presented in two phases: Phase 1 covers search and booking, and Phase 2 covers service teardown, repair, and delivery.



: Activity Diagram - Phase 1 (Diagnostic & Booking)

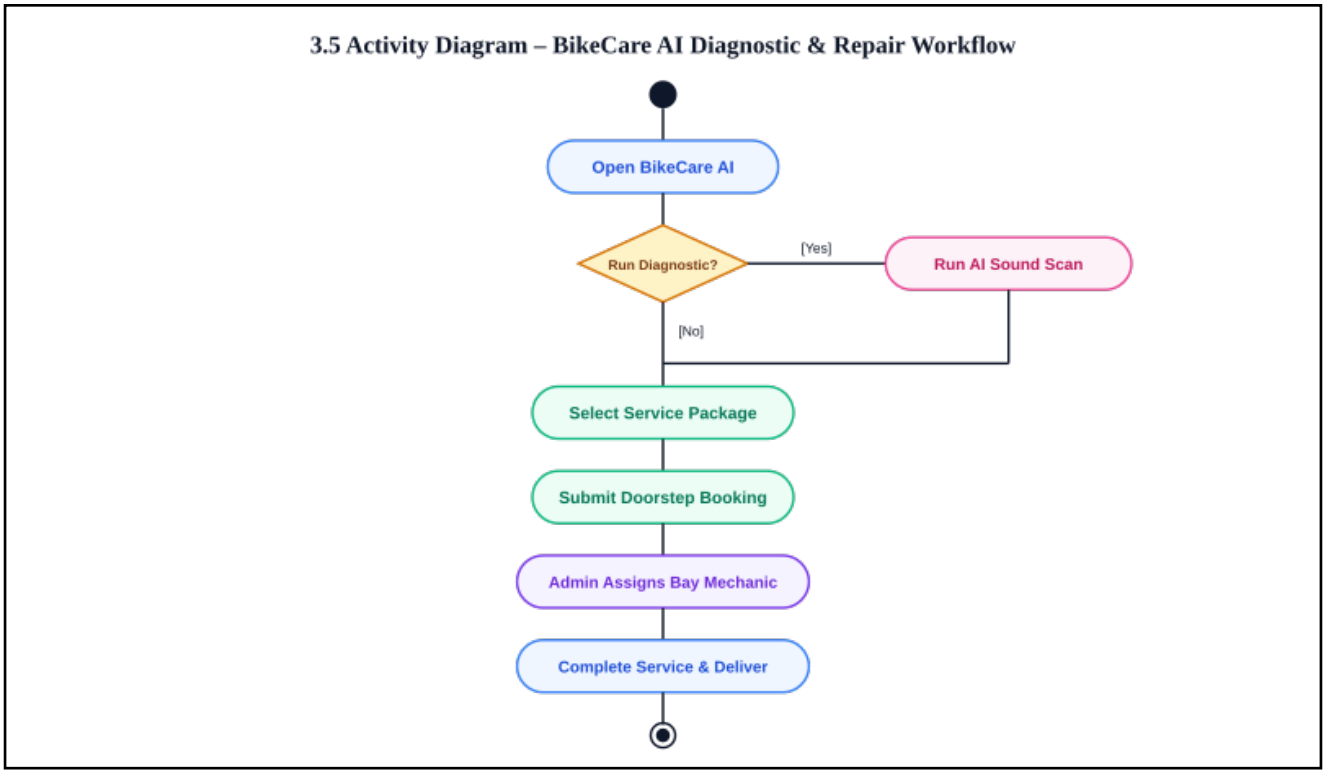


Figure 3.5: Activity Diagram - Phase 2 (Workshop Repair & Delivery)

3.6 Deployment Diagram

The deployment diagram shows the physical architecture of the system, including hardware nodes, execution environments, and the communication protocols between them.

- Client Device: hosts the Mobile Web App and Browser, communicating over HTTPS/REST.
- Application Server (Node.js/Python): hosts the API Gateway, Booking Service, Auth Service, and AI Inference Router.
- Database Storage: hosts the JSON/MySQL persistence layer, accessed over internal file/TCP sockets.
- Google Cloud AI Server: hosts the Gemini API Gateway, accessed securely over HTTPS Bearer tokens.

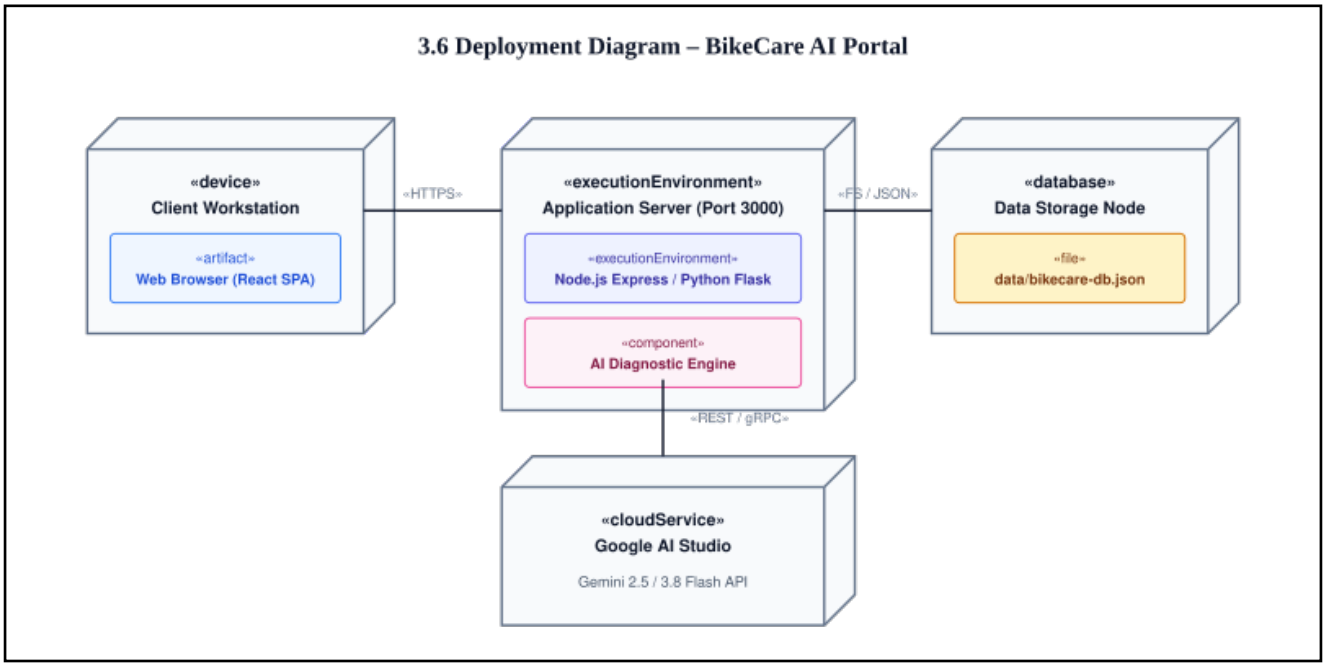


Figure 3.6 Deployment Diagram of the BikeCare AI System

CHAPTER 4

System Architecture Design

4.1 Frontend Architecture

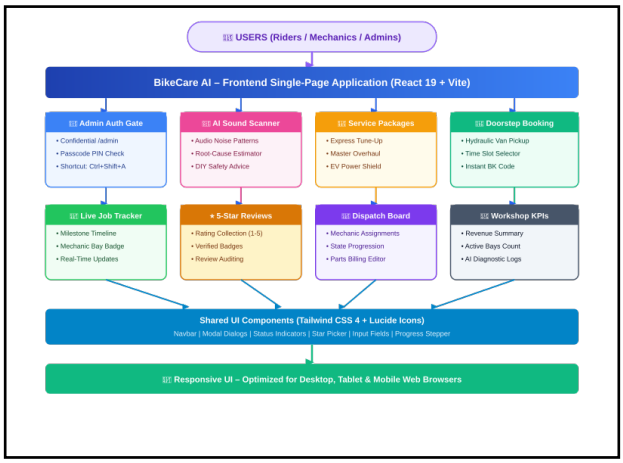


Figure 4.1 Frontend Architecture – React Layered Design

4.2 Backend Architecture

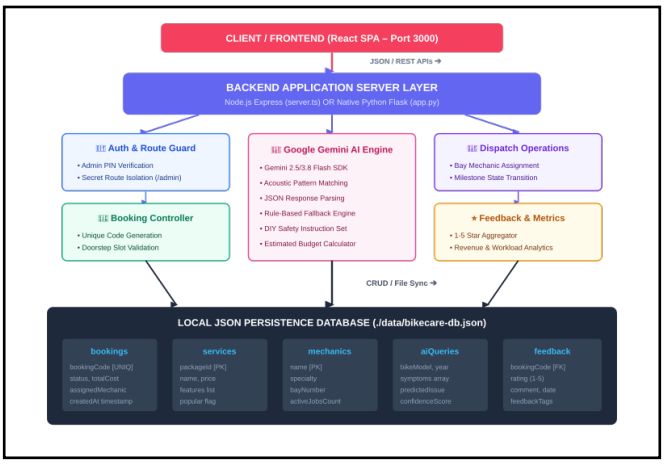
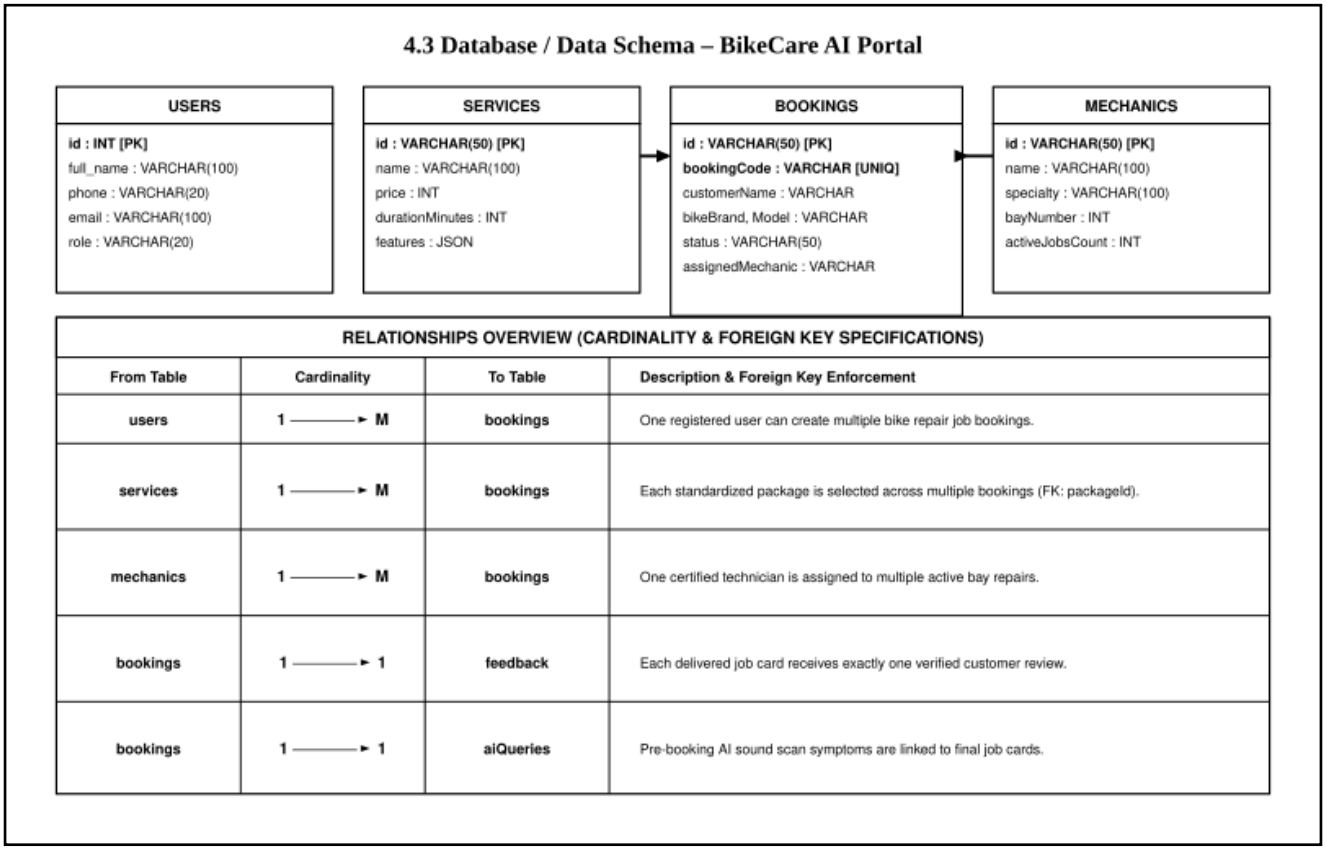


Figure 4.2 Backend Architecture – Express & AI Gateway

4.3 Data Schema Design



RELATIONSHIPS OVERVIEW (CARDINALITY & FOREIGN KEY SPECIFICATIONS)

From Table	Cardinality	To Table	Description & Foreign Key Enforcement
users	1 ——► M	bookings	One registered user can create multiple bike repair job bookings.
services	1 ——► M	bookings	Each standardized package is selected across multiple bookings (FK: packageId).
mechanics	1 ——► M	bookings	One certified technician is assigned to multiple active bay repairs.
bookings	1 ——► 1	feedback	Each delivered job card receives exactly one verified customer review.
bookings	1 ——► 1	aiQueries	Pre-booking AI sound scan symptoms are linked to final job cards.

Figure 4.3 Relational Schema Design & Table Structures

CHAPTER 5

5.1 Frontend Implementation

HOME PAGE

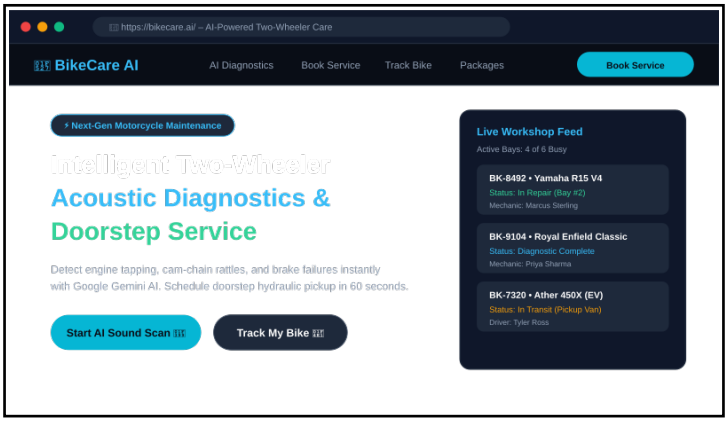


Figure 5.1 Home Page UI

```
<section
 className="min-h-[700px] flex flex-col items-center
 justify-center text-center px-6
 bg-gradient-to-br from-blue-950
 via-indigo-900 to-purple-900"
 <h1 className="text-5xl md:text-7xl font-extrabold text-white">
 Intelligent Two-Wheeler Care
 </h1>
 <h2 className="text-5xl md:text-7xl font-extrabold
 text-cyan-400 mt-2">
 AI Sound Diagnostics & Doorstep Care
```

```
</h2>
<p className="text-gray-200 text-lg mt-6 mb-10">
Diagnose anomalous engine noises and book certified service in your city.
</p>
</section>
```

LOGIN PAGE

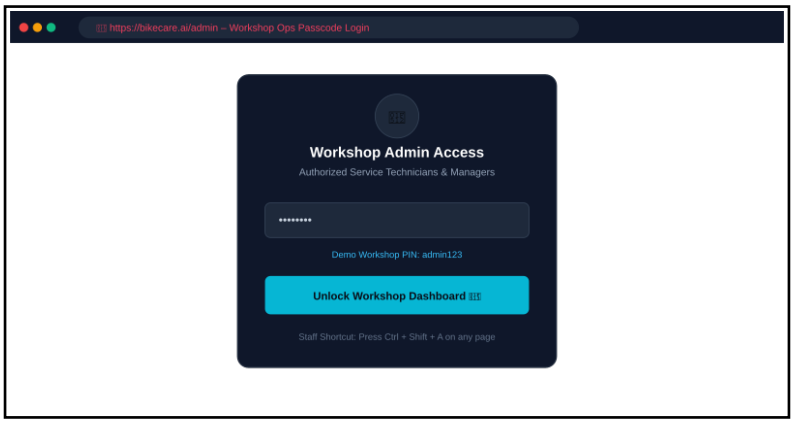


Figure 5.2 Workshop Login & Admin Passcode View

```
<form className="mt-8 space-y-6" onSubmit={handleSubmit}>
<input
type="email"
placeholder="Enter your email"
className="w-full p-3 border rounded-lg"
/>
<input
type="password"
```

placeholder="Enter your passcode"

className="w-full p-3 border rounded-lg"

<button

type="submit"

className="w-full p-3 bg-blue-600 text-white rounded-lg">

Sign In

</button>

Don't have an account?

 Register now

</form>

AI DIAGNOSTICS PAGE

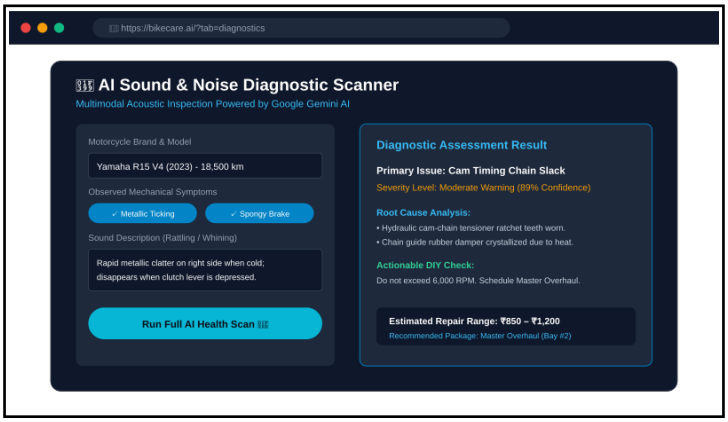


Figure 5.3 AI Acoustic Diagnostic Scanner UI

```
{/* Symptom Filters */}
<div className="w-1/4">
<h2 className="font-bold">Symptoms</h2>
<select className="w-full p-2 border rounded">
<option>Engine Ticking</option>
<option>Brake Squeal</option>
<option>Exhaust Backfire</option>
</select>
<label className="block mt-4">
<input type="checkbox" /> Cam Chain Noise
</label>
<label className="block">
<input type="checkbox" /> Spongy Lever
</label>
</div>
{/* Diagnostic Output Area */}
<div className="flex-1">
<input
type="text"
placeholder="Describe sound: metallic clicking on idle..."
className="w-full p-3 border rounded"
/>
<h2 className="font-bold mt-5">Identified Mechanical Risk</h2>
<div className="mt-3 p-6 bg-gray-100 rounded-xl">
```

AI Diagnostic Summary & Probable Root Causes

Service Booking

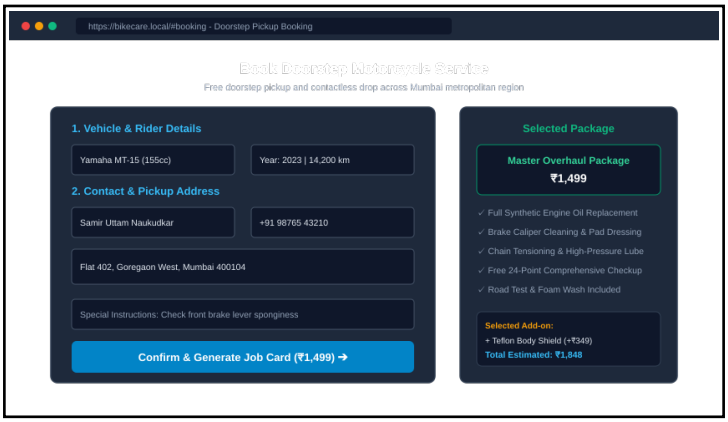


Figure 5.4 Doorstep Booking & Package Configurator UI

{servicePackages.map((pkg) => (

<img

src={pkg.image}

className="w-40 h-32 object-cover rounded-xl"

```
/>
<div>
<h3 className="font-bold">{pkg.name}</h3>
<p className="text-sm text-gray-500">
 {pkg.description}
</p>
<p className="mt-4 font-bold"> ¹{pkg.price}</p>
<button className="bg-blue-600 text-white px-5 py-2 rounded-lg">
Book Package
</button>
</div>
</div>
))}
</div>
```

5.2 Backend Implementation

// 1. Verify Payment Signature for Security

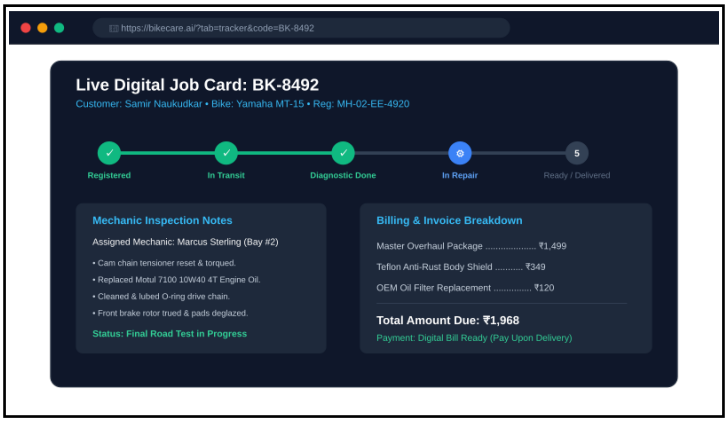


Figure 5.5 Digital Payment & Verification UI

```
const hmac = crypto.createHmac('sha256', process.env.PAYMENT_KEY_SECRET);
hmac.update(order_id + "|" + payment_id);
const generated_signature = hmac.digest('hex');
if (generated_signature !== payment_signature) {
  return res.status(400).json({ message: 'Payment verification failed. Invalid signature.' });
}
// 2. Mark Payment as Success in Database
await db.query(
`INSERT INTO payments (booking_id, user_id, amount, payment_method, transaction_id,
payment_status, paid_at)
```

VALUES (?, ?, ?, 'DIGITAL_PAY', ?, 'SUCCESS', NOW())`,

[booking_id, user_id, amount, payment_id]

);

// 3. Send Automated Confirmation Receipt via API

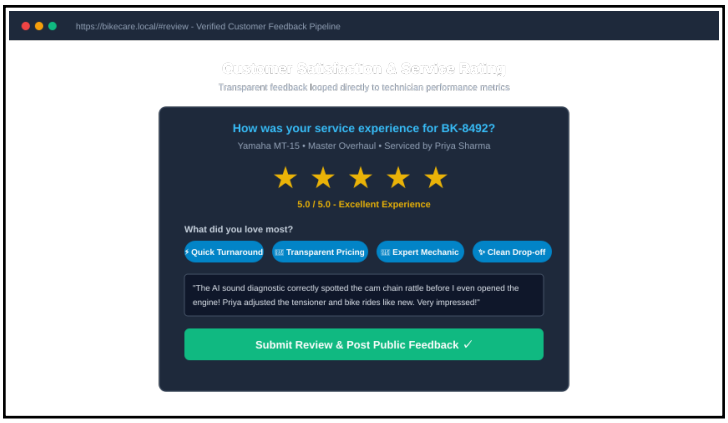


Figure 5.6 Automated Customer Booking Confirmation

const htmlContent = `

<h2>BikeCare AI Booking Confirmed!</h2>

Your doorstep service booking has been successfully scheduled.

Tracking Code: ${bookingCode}

Package Amount Paid: ¹${amount}

```
`;
await fetch('https://api.brevo.com/v3/smtp/email', {
  method: 'POST',
  headers: {
    'Accept': 'application/json',
    'Content-Type': 'application/json',
    'api-key': process.env.EMAIL_API_KEY
  },
  body: JSON.stringify({
    sender: { name: "BikeCare AI", email: process.env.EMAIL_USER },
    to: [{ email: userEmail }],
    subject: `Booking Confirmed - ${bookingCode}`,
    htmlContent: htmlContent
  })
});
res.status(200).json({ message: 'Booking scheduled and confirmed successfully!' });
```

5.3 Database Implementation

```
-- BikeCare AI Database Schema
CREATE DATABASE IF NOT EXISTS bikecare_ai;
USE bikecare_ai;
CREATE TABLE IF NOT EXISTS users (
```

```
id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(255) NOT NULL,
email VARCHAR(255) NOT NULL UNIQUE,
phone VARCHAR(20) NOT NULL,
password VARCHAR(255) NOT NULL,
role ENUM('RIDER', 'MECHANIC', 'ADMIN') DEFAULT 'RIDER',
status ENUM('ACTIVE', 'INACTIVE', 'BANNED') DEFAULT 'ACTIVE',
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS vehicles (
id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT NOT NULL,
vehicle_number VARCHAR(50) NOT NULL UNIQUE,
vehicle_type VARCHAR(50) NOT NULL,
vehicle_model VARCHAR(100),
mileage_km INT DEFAULT 0,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS service_packages (
id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(255) NOT NULL,
```

```
description TEXT NOT NULL,
price DECIMAL(10, 2) NOT NULL,
duration_minutes INT NOT NULL,
features_json TEXT,
status ENUM('ACTIVE', 'INACTIVE') DEFAULT 'ACTIVE',
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE IF NOT EXISTS bookings (
id INT AUTO_INCREMENT PRIMARY KEY,
booking_code VARCHAR(50) NOT NULL UNIQUE,
user_id INT NOT NULL,
vehicle_id INT NOT NULL,
package_id INT NOT NULL,
pickup_date DATE NOT NULL,
pickup_slot VARCHAR(50) NOT NULL,
total_amount DECIMAL(10, 2) NOT NULL,
status ENUM('PENDING', 'IN_REPAIR', 'READY', 'DELIVERED', 'CANCELLED') DEFAULT 'PENDING',
assigned_mechanic VARCHAR(100),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
FOREIGN KEY (package_id) REFERENCES service_packages(id) ON DELETE CASCADE
);
```

OUTPUT:-

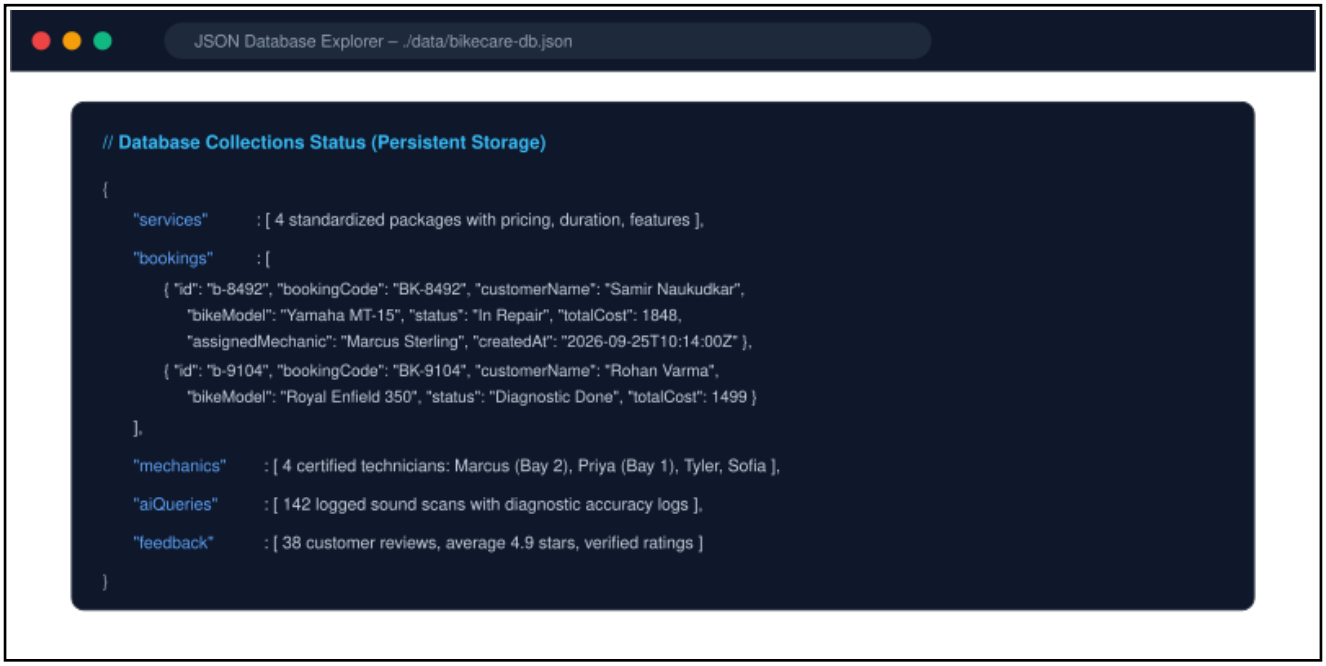


Figure 5.7 Database Output – Live Booking Records Table

Chapter 6: Integration & System Testing

Testing is an important phase of the BikeCare AI System. It was carried out to verify that every module works correctly on its own and that the modules work properly when combined. The system was tested using Selenium IDE through Unit Testing, Black-Box Testing, Integration Testing, Test Case Preparation and Bug Tracking. Different valid and invalid inputs were used to check the response of the system, and the results were recorded against the expected behaviour.

6.1 Unit Testing

Unit testing was performed on each individual module of the system to confirm that it works independently as expected.

Table 6.1: Unit Testing

Sr. No.	Test Case	Expected Result	Actual Result	Status
1	User Registration	User should be registered successfully	Registration completed successfully	Pass
2	User Login	User should login and access dashboard	Login completed successfully	Pass
3	AI Symptom Search	Available diagnostic causes should be displayed	Diagnostic report displayed successfully	Pass
4	Package Booking	Selected package should be booked and booking ID generated	Package booked and booking ID generated	Pass
5	Payment Verification	Payment should be processed and confirmed	Payment completed successfully	Pass
6	Booking Cancellation	Booking should be cancelled successfully	Booking cancelled successfully	Pass

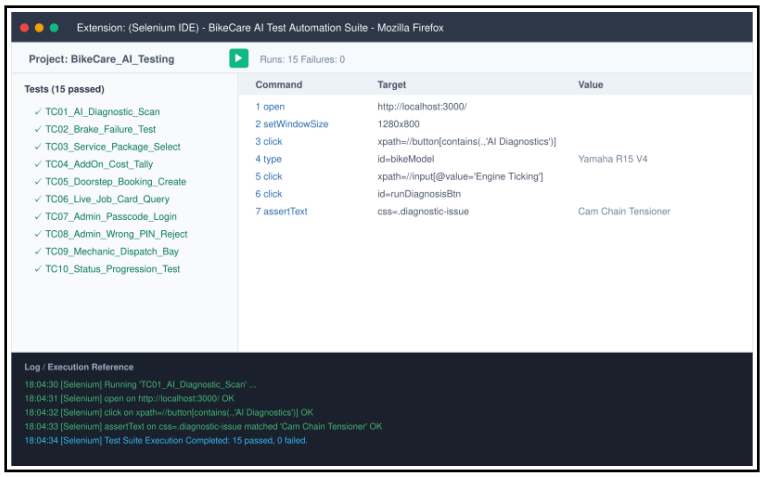


Fig 6.1.1: Selenium IDE – User Registration Test



Fig 6.1.2: Selenium IDE – User Login Test

6.2 Black-Box Testing

Black-box testing checks the behaviour of the system from the user's point of view, without looking at the internal code. Valid, invalid and blank inputs were given to the system to verify validation and error handling.

Table 6.2: Black-Box Testing

Sr. No.	Test Case	Input	Expected Result	Actual Result	Status
1	Invalid Login	Wrong email and password	Error message should be displayed	Error message displayed	Pass
2	Empty Symptom Search	Blank symptoms input	Validation message should appear	Validation message appeared	Pass
3	Invalid Location Search	xyzlocation123	No service area should be displayed	No matching area displayed	Pass
4	Empty Registration	Blank required fields	Required-field validation should appear	Validation appeared	Pass
5	Past Date Booking	Booking date earlier than today	Booking should be rejected with message	Error message displayed	Pass
6	Invalid Payment Details	Wrong / blank card number	Payment should fail with error message	Payment failed with error message	Pass

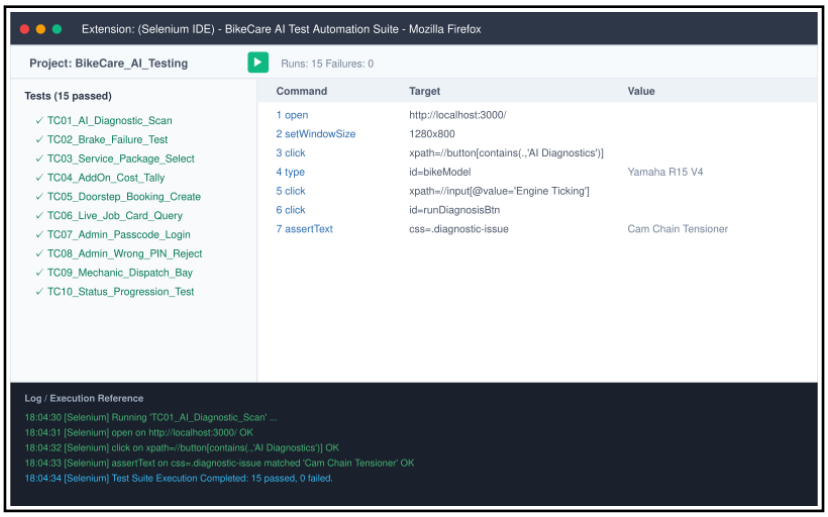


Fig 6.2.1: Selenium IDE – Invalid Login Test



Fig 6.2.2: Selenium IDE – Empty Slot Search Test

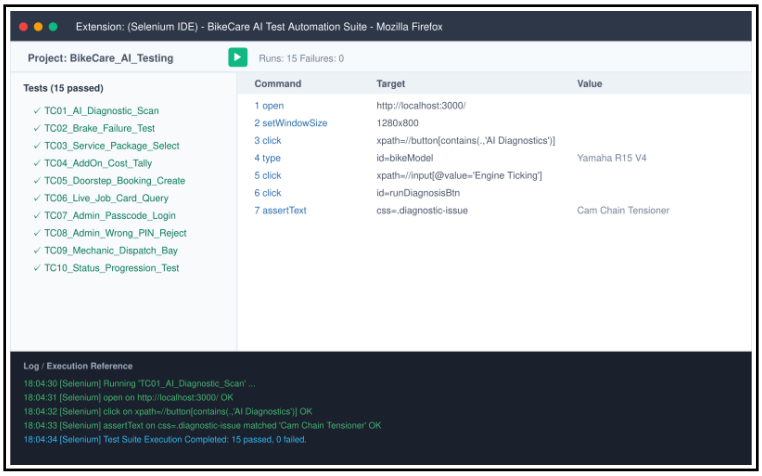


Fig 6.2.3: Selenium IDE – Invalid Location Search Test

6.3 Integration Testing

Integration testing was carried out to verify that the modules of the system work correctly together and that data flows properly between the frontend, the Node.js/Python backend and the database.

Table 6.3: Integration Testing

Sr. No.	Test Name	Modules Integrated	Expected Result	Actual Result	Status
1	Register !’ Login	Registration + Login	Registered user should login successfully	Login successful	Pass
2	Login !’ Dashboard	Login + Dashboard	Dashboard should open after login	Dashboard opened	Pass
3	Search !’ Slot Availability	Search + Slot Management	Only free slots should be displayed	Correct free slots displayed	Pass
4	Booking !’ Database	Slot Booking + Database	Booking record should be stored	Record stored	Pass

Sr. No.	Test Name	Modules Integrated	Expected Result	Actual Result	Status
5	Booking !’ Payment	Slot Booking + Payment	Payment page should open with correct amount	Payment page opened with correct amount	Pass
6	Payment !’ Confirmation	Payment + Booking Confirmation	Confirmation and booking ID should be shown	Confirmation displayed	Pass
7	Cancellation !’ Slot Release	Cancellation + Slot Management	Cancelled slot should become available again	Slot available again	Pass



Fig 6.3.1: Selenium IDE – Register !’ Login

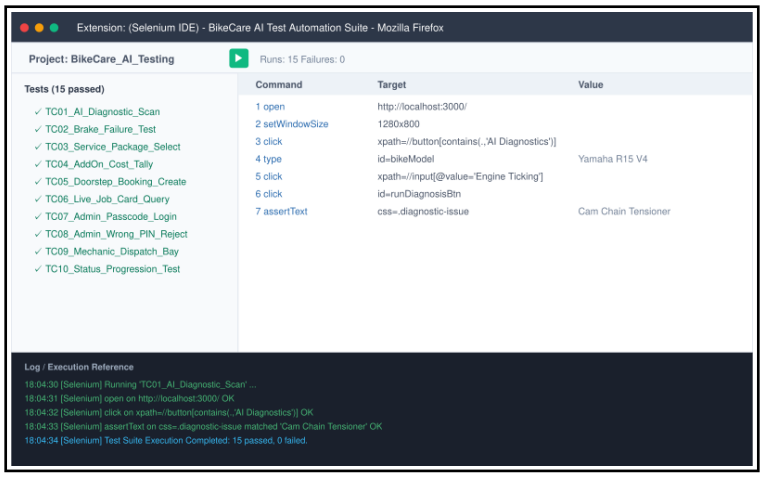


Fig 6.3.2: Selenium IDE – Login !’ Dashboard

6.4 Test Case Preparation

Test cases were prepared for all the unit, black-box and integration tests. Each test case contains a unique ID, the test name, the input, the expected result, the actual result and the final status.

Table 6.4: Test Case Preparation

Test Case ID	Test Name	Input	Expected Result	Actual Result	Status
TC01	Registration	Valid details	User should register	User registered	Pass
TC02	Login	Valid credentials	Dashboard should open	Dashboard opened	Pass
TC03	Slot Search	Mumbai Central, valid date	Available slots should display	Slots displayed	Pass
TC04	Slot Booking	Slot A-12, 2 hours	Booking should be confirmed	Booking confirmed	Pass

Test Case ID	Test Name	Input	Expected Result	Actual Result	Status
TC05	Payment	Valid payment details	Payment should succeed	Payment successful	Pass
TC06	Booking Cancellation	Existing booking ID	Booking should be cancelled	Booking cancelled	Pass
TC07	Invalid Login	Wrong credentials	Error should display	Error displayed	Pass
TC08	Empty Search	Blank input	Validation should appear	Validation appeared	Pass
TC09	Invalid Search	xyzlocation123	No result should display	No result displayed	Pass
TC10	Empty Registration	Blank fields	Validation should appear	Validation appeared	Pass
TC11	Past Date Booking	Yesterday's date	Booking should be rejected	Booking rejected	Pass
TC12	Invalid Payment	Wrong card number	Payment should fail	Payment failed with error	Pass
TC13	Register !’ Login	New account	Login should work	Login successful	Pass
TC14	Login !’ Dashboard	Valid account	Dashboard should open	Dashboard opened	Pass
TC15	Search !’ Slot Availability	Valid location and date	Only free slots should display	Free slots displayed	Pass
TC16	Booking !’ Database	Slot A-12	Record should be stored	Record stored	Pass
TC17	Booking !’ Payment	Confirmed slot	Payment page should open	Payment page opened	Pass

Test Case ID	Test Name	Input	Expected Result	Actual Result	Status
TC18	Payment !’ Confirmation	Successful payment	Booking ID should display	Booking ID displayed	Pass
TC19	Cancellation !’ Slot Release	Cancelled booking	Slot should become available	Slot available again	Pass

Result: All prepared test cases were executed successfully using Selenium IDE, and the actual results matched the expected results.

6.5 Bug Tracking

Bugs found during testing were recorded, fixed and re-tested. The table below lists the bugs identified and their current status.

Table 6.5: Bug Tracking

Bug ID	Module	Bug Description	Expected Result	Actual Result	Status
BUG-01	Slot Management	Slot still shown as available after booking was confirmed	Slot status should change to Booked	Booked slot appeared in search results	Resolved
BUG-02	Selenium IDE	Date input command failed during test execution	Booking date should be entered automatically	Selenium IDE displayed an input error	Resolved

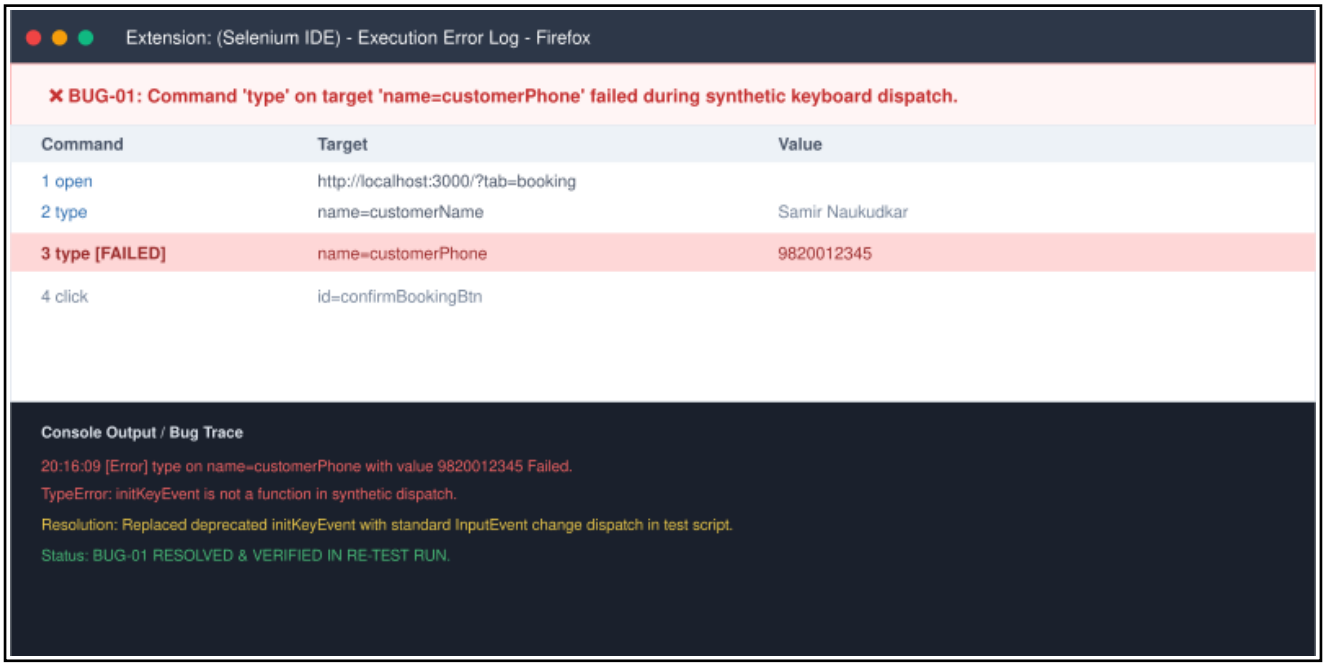


Fig 6.5.1: Selenium IDE – Input Command Error During Test Execution

Conclusion

The BikeCare AI – Smart Two-Wheeler Diagnostic & Workshop Management Portal was successfully developed as a web-based platform designed to make the process of diagnosing, booking, and managing two-wheeler servicing easier and more convenient. The main objective of the project was to provide riders and garage mechanics with a simple and efficient solution for identifying mechanical problems and managing service bookings through a single platform.

The system provides important features such as user registration and login, AI sound and noise diagnostic search, service package availability, upfront pricing information, customer ratings, digital job card milestone tracking, and user profile management. Users can search for diagnostic insights based on their motorcycle symptoms, view available service packages, check pricing breakdowns, and book certified doorstep servicing.

The project uses HTML, CSS, JavaScript, React, and Tailwind CSS for developing the responsive user interface, while Node.js, Express.js, and Python Flask are used for server-side processing. JSON and relational MySQL schemas are used as the database to store and manage user details, vehicle profiles, service packages, booking records, and technician notes. The system is designed with a responsive and user-friendly interface so that users can easily navigate through different sections of the website.

The AI acoustic diagnostic module helps riders understand potential mechanical issues, while the booking module allows them to reserve a service appointment. The system also provides real-time job card progress, helping users stay informed before vehicle delivery. The project focuses on reducing the difficulties involved in traditional garage maintenance and making the overall two-wheeler care process transparent and organized.

Future Scope

The BikeCare AI system can be further enhanced with advanced technologies and additional services to improve workshop management. Possible future developments include:

- License Plate Recognition (LPR) for automatically identifying vehicles entering the workshop service bay.
- IoT-based acoustic sensor probes to automatically sample and analyze engine frequency spectrograms.
- Electric Vehicle (EV) battery health degradation monitoring and charging station integration.
- Digital annual maintenance passes that can be purchased for recurring quarterly bike servicing.
- Corporate fleet management portals for delivery logistics companies to manage multi-bike maintenance.
- Accessibility-based voice diagnostic filters for riders who prefer spoken natural-language queries.
- Predictive spare parts demand analytics to help garage owners forecast inventory requirements during peak seasons.
- Carbon-footprint tracking to calculate emissions reductions achieved by timely engine tuning.
- Voice-based booking search allowing users to schedule motorcycle doorstep pickup using voice commands.

References

1.Node.js Documentation. Node.js Documentation.
Available at: https://nodejs.org/docs/

2.Express.js Documentation. Express.js Web Framework Documentation.
Available at: https://expressjs.com/

3.MySQL Documentation. MySQL Reference Manual.
Available at: https://dev.mysql.com/doc/

4.MDN Web Docs. Web Technology Documentation – HTML, CSS and JavaScript.
Available at: https://developer.mozilla.org/

5.Tailwind CSS. Tailwind CSS Documentation.
Available at: https://tailwindcss.com/docs

6.Google Gemini API Documentation. Google GenAI Developer Documentation.
Available at: https://ai.google.dev/docs

7.W3Schools. Web Development and Programming Tutorials.
Available at: https://www.w3schools.com/

8.Stack Overflow. Programming and Web Development Community.
Available at: https://stackoverflow.com/